

Module : Bases de Données Avancées

Rapport

Application de Gestion de Pharmacie

Réalisé par :

EL ABBASSI MOHAMED TAHA

Encadré par :

Prof. HRIMECH

1 Introduction

Cette application de bureau permet la gestion complète d'une pharmacie. Développée en Java avec une interface graphique JavaFX, elle utilise Hibernate comme framework ORM pour la persistance des données et MySQL 8 comme système de gestion de base de données relationnelle.

L'application propose deux niveaux d'accès distincts : un rôle Administrateur disposant de tous les droits, et un rôle Employé avec un accès restreint en consultation.

Objectifs principaux :

- Gérer le stock des médicaments : ajout, modification et suppression
- Alerter automatiquement lorsque le stock est inférieur à 10 unités
- Suivre les dates d'expiration des médicaments
- Offrir un système d'authentification sécurisé avec deux rôles
- Permettre la recherche rapide d'un médicament par nom
- Permettre à l'administrateur de gérer les comptes employés

2 Base de Données MySQL

2.1 Création de la base de données

La base de données pharmacie est créée dans MySQL 8. Elle contient deux tables principales : `medicament` et `utilisateur`.

```
1 CREATE DATABASE pharmacie;
2 USE pharmacie;
3
4 -- Table des médicaments
5 CREATE TABLE medicament (
6     id          INT PRIMARY KEY AUTO_INCREMENT,
7     nom         VARCHAR(100),
8     description VARCHAR(255),
9     prix        DOUBLE,
10    quantite    INT,
11    date_expiration DATE
12 );
13
14 -- Table des utilisateurs
15 CREATE TABLE utilisateur (
16     id          INT PRIMARY KEY AUTO_INCREMENT,
17     username    VARCHAR(50),
18     password    VARCHAR(50),
19     role        VARCHAR(20) -- valeurs : ADMIN ou EMPLOYE
20 );
21
22 -- Insertion des comptes par défaut
23 INSERT INTO utilisateur (username, password, role)
24 VALUES ('admin', 'admin123', 'ADMIN');
25
26 INSERT INTO utilisateur (username, password, role)
27 VALUES ('employee1', 'emp123', 'EMPLOYE');
```

2.2 Description des tables

Table `medicament` :

- `id` : clé primaire auto-incrémentée
- `nom` : nom commercial du médicament
- `description` : description et usage
- `prix` : prix unitaire en dirhams
- `quantite` : quantité disponible en stock

- `date_expiration` : date limite d'utilisation

Table utilisateur :

- id : clé primaire auto-incrémentée
- username : identifiant de connexion

- password : mot de passe
- role : ADMIN ou EMPLOYE

2.3 Mapping Hibernate — Entité Medicament

La classe `Medicament` est annotée avec les annotations JPA pour le mapping automatique avec la table MySQL :

```

1 @Entity
2 @Table(name = "medicament")
3 public class Medicament {
4     @Id
5     @GeneratedValue(strategy = GenerationType.IDENTITY)
6     private int id;
7     @Column(name = "nom")
8     private String nom;
9     @Column(name = "description")
10    private String description;
11    @Column(name = "prix")
12    private double prix;
13    @Column(name = "quantite")
14    private int quantite;
15    @Column(name = "date_expiration")
16    private Date dateExpiration;
17    // getters et setters...
18 }

```

3 Configuration Hibernate

3.1 Fichier hibernate.cfg.xml

Le fichier de configuration Hibernate définit la connexion à MySQL 8, le dialecte SQL et le mapping des entités :

```

1 <hibernate-configuration>
2   <session-factory>
3     <property name="hibernate.dialect">
4       org.hibernate.dialect.MySQL5Dialect</property>
5     <property name="hibernate.connection.driver_class">
6       com.mysql.cj.jdbc.Driver</property>
7     <property name="hibernate.connection.url">
8       jdbc:mysql://localhost:3306/pharmacie?useSSL=false
9       &serverTimezone=UTC
10      &allowPublicKeyRetrieval=true</property>
11     <property name="hibernate.connection.username">root</property>
12     <property name="hibernate.connection.password">root123</property>
13     <property name="hibernate.show_sql">true</property>
14     <mapping class="entities.Medicament"/>
15     <mapping class="entities.Utilisateur"/>
16   </session-factory>
17 </hibernate-configuration>

```

4 Fonctionnalités Principales

4.1 Authentification et gestion des rôles

La page de login vérifie les identifiants via une requête HQL Hibernate et redirige l'utilisateur selon son rôle :

```

1 public Utilisateur login(String username, String password) {
2     Session session = HibernateUtil.getSessionFactory().openSession();
3     Utilisateur u = (Utilisateur) session.createQuery(
4         "from Utilisateur where username=:u and password=:p")

```

```

5         .setParameter("u", username)
6         .setParameter("p", password)
7         .uniqueResult();
8     session.close();
9     return u;
10 }
11 String fxml = u.getRole().equals("ADMIN") ?
12     "/vue/admin.fxml" : "/vue/employe.fxml";

```

4.2 Opérations CRUD sur les médicaments

```

1 public void ajouter(Medicament m) {
2     Session session = HibernateUtil.getSessionFactory().openSession();
3     Transaction tx = session.beginTransaction();
4     session.save(m); tx.commit(); session.close();
5 }
6 public void modifier(Medicament m) {
7     Session session = HibernateUtil.getSessionFactory().openSession();
8     Transaction tx = session.beginTransaction();
9     session.update(m); tx.commit(); session.close();
10 }
11 public void supprimer(int id) {
12     Session session = HibernateUtil.getSessionFactory().openSession();
13     Transaction tx = session.beginTransaction();
14     Medicament m = (Medicament) session.get(Medicament.class, id);
15     if (m != null) session.delete(m);
16     tx.commit(); session.close();
17 }

```

4.3 Recherche par nom et alerte stock faible

```

1 // Recherche insensible a la casse
2 public List<Medicament> rechercherParNom(String nom) {
3     Session session = HibernateUtil.getSessionFactory().openSession();
4     List<Medicament> liste = session.createQuery(
5         "from Medicament where lower(nom) like :nom")
6         .setParameter("nom", "%" + nom.toLowerCase() + "%").list();
7     session.close();
8     return liste;
9 }
10 // Alerte stock faible
11 private void verifierStockFaible() {
12     long count = service.listerTous()
13         .stream().filter(m -> m.getQuantite() < 10).count();
14     if (count > 0)
15         alerteLabel.setText("ALERTE: " + count +
16             " medicament(s) avec stock faible (< 10)");
17 }

```

4.4 Droits selon le rôle

Fonctionnalité	Admin	Employé
Ajouter / Modifier / Supprimer		×
Rechercher par nom		
Voir stock faible		
Voir dates d'expiration		
Actualiser la liste		
Gérer les employés		×
Déconnexion		

5 Conclusion

L'application de gestion de pharmacie répond à tous les besoins fonctionnels définis : authentification par rôles, gestion complète du stock des médicaments avec les dates d'expiration, alertes automatiques pour le stock faible, recherche rapide et gestion des employés par l'administrateur.

Ce projet a permis de mettre en pratique les concepts fondamentaux du développement Java d'entreprise, notamment la persistance des données avec Hibernate et MySQL, la création d'interfaces graphiques avec JavaFX, et l'organisation du code selon l'architecture MVC.

Améliorations futures envisagées :

- Chiffrement des mots de passe avec BCrypt
- Export de la liste des médicaments en PDF
- Tableau de bord avec statistiques et graphiques
- Notifications automatiques pour les médicaments proches de la date d'expiration